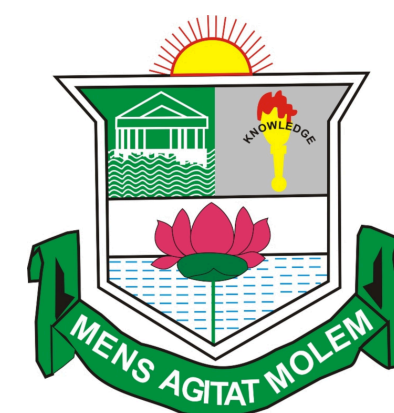




**CHELLAMAL WOMEN'S
COLLEGE, GUINDY**



**B.Sc. Computer Science with AI
PROJECT DOCUMENTATION**

EduGenie

Google Gemini Powered Learning Assistant

A Lightweight AI-Powered Educational Assistant

TEAM MEMBERS	
TEAM LEADER	R. Manimagalai
TEAM MEMBERS	M. Keerthana M. Jeevalakshmi M. Paveena M. Nivetha

ACADEMIC YEAR • 2025–28

Table of Contents

1. Project Overview
2. Objectives and Use Scenarios
3. System Architecture
4. Prerequisites and Environment Setup
5. Project Workflow and Milestones
6. Functional Modules
7. Backend API Design
8. Frontend Development and Integration
9. Deployment and Execution
10. Functional Testing
11. User Interaction Flow
12. Challenges and Learning Outcomes

- 13. Conclusion
- 14. Future Enhancements

1. Project Overview

EduGenie is a lightweight AI-powered educational assistant designed to simplify learning through generative AI. It provides students and self-learners with smart question answering, simplified concept explanations, quiz generation, text summarization, and personalized learning recommendations. The system combines a FastAPI backend with a simple HTML/CSS web interface and uses cloud-based and lightweight local AI models.

The project is designed to remain accessible on modest hardware while still providing useful AI-powered learning support. The architecture separates each learning capability into an independent module, making the application easier to maintain and extend.

2. Objectives and Use Scenarios

2.1 Core Objectives

- Provide concise and useful answers to academic and general knowledge questions.
- Explain complex concepts in simpler, learner-friendly language.
- Generate topic- or passage-based multiple-choice quizzes.
- Summarize long educational passages for quick revision.
- Create structured learning paths from beginner to advanced level.
- Provide a lightweight web interface backed by modular REST APIs.

2.2 Representative Use Scenarios

Scenario	Description
Question Answering	A learner asks a question such as “Which is the largest ocean?” and receives an AI-generated answer.
Quiz Generation	A learner studying the Pythagoras Theorem selects the quiz function to assess understanding.
Learning Path	A learner exploring SQL requests a structured path containing beginner-to-advanced topics, timelines, and learning resources.

3. System Architecture

EduGenie follows a modular client-server architecture. The frontend collects the learner’s task and input, the FastAPI backend routes the request to the appropriate module, and the selected AI model generates the response.

Figure 1. System architecture / project structure from the source document.

3.1 Technology Components

Component	Purpose
Python 3.10+	Primary programming language and runtime.
FastAPI	Backend web framework used to expose RESTful endpoints.
Google Gemini 1.5 Pro	Cloud AI model used for Q&A, summarization, quiz generation, and learning paths.
LaMini-Flan-T5-783M	Lightweight local model used for simplified concept explanation.
HTML&CSS	Frontend structure, forms, and responsive styling.
Uvicorn	ASGI server used to run the FastAPI application.
Jinja2	HTML templating engine used by the web interface.

4. Prerequisites and Environment Setup

The source project document specifies the following prerequisites and setup components.

- Python 3.10 or later.
- FastAPI framework.
- Basic HTML and CSS knowledge for the templates and static files.
- A Google Gemini API key.
- Uvicorn ASGI server.
- Jinja2 HTML templating engine.

4.1 Setup Procedure

1. Install Python 3.10+ and verify the installation using: `python --version`
2. Install Uvicorn using: `pip install uvicorn`
3. Install Jinja2 using: `pip install jinja2`
4. Create/access a Google AI Studio project, enable the Gemini API, generate an API key, and store it securely.
5. Install the remaining Python dependencies listed in `requirements.txt`.
6. Keep the API key outside publicly shared source code and configure it through the project’s runtime/environment configuration.

5. Project Workflow and Milestones

Milestone 1 – Model Selection and Architecture

Select the AI models and define the modular folder architecture.

Milestone 2 – Core Functionalities Development

Implement explanation, Q&A, quiz, summary, learning-path modules and connect them through FastAPI.

Milestone 3 – Frontend Development

Build the HTML/CSS interface and integrate form submission with backend APIs.

Milestone 4 – Deployment

Run the application locally and perform functional tests across all supported tasks.

5.1 Folder Architecture

File / Path	Responsibility
main.py	FastAPI application and API routing
explanation_module.py	Concept explanation logic
qna.py	Question-answering logic
quiz_module.py	Quiz generation logic
summary_module.py	Summarization logic
learning_path.py	Learning recommendation logic
templates/index.html	HTML frontend
static/style.css	Frontend styling
requirements.txt	Python dependencies

6. Functional Modules

6.1 Explanation Module

The explanation module uses LaMini-Flan-T5-783M, a lightweight instruction-tuned model, to provide concise, context-aware explanations. Its purpose is to break complex concepts into readable language for beginners, school students, and self-learners.

6.2 Q&A Module

The Q&A module uses Gemini 1.5 Pro to handle general knowledge and academic questions. It uses the model’s contextual capabilities to produce AI-driven responses across diverse educational topics.

6.3 Quiz Module

The quiz module generates three multiple-choice questions from a supplied passage. Each question contains four options and a correct answer. The response is expected in JSON format, which simplifies integration with the application. Markdown code blocks are cleaned before parsing; generation or parsing failures are returned as useful error information.

6.4 Summary Module

The summary module uses Gemini to convert long educational paragraphs into concise versions for quick revision. The intended behavior is to retain key information while removing redundancy.

6.5 Learning Path Module

The learning-path module uses Gemini to create a structured path from beginner to advanced concepts. The generated plan can include difficulty progression and resources such as videos, articles, or books, while adapting to the learner’s level.

7. Backend API Design

The FastAPI backend exposes RESTful endpoints, with each endpoint connected to the corresponding module logic.

Endpoint	Function
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Personalized learning recommendations

The API layer acts as the bridge between the web interface and the AI processing modules, keeping the application modular and easier to extend.

8. Frontend Development and Integration

8.1 Web Interface

- Task dropdown containing Explain, QnA, Quiz, Summary, and Recommend Path.
- Text area for learner input.
- Submit button for sending the selected task.
- Responsive CSS styling for buttons, input areas, and result containers.

8.2 Live Integration

When the learner submits the form, the frontend sends a POST request to the FastAPI backend. The generated result is then displayed below the input area, providing an interactive learning experience.

9. Deployment and Execution

9.1 Run Locally

Start the FastAPI application using the command below:

```
uvicorn main:app --reload
```

Then open the local application at:

9.2 Execution Flow

- Launch the FastAPI application with Uvicorn.
- Open the EduGenie web interface in a browser.
- Select a learning task from the dropdown.
- Enter the question, topic, or passage.
- Submit the request.
- FastAPI routes the request to the selected module.
- The module invokes the configured AI model and processes the response.
- The result is displayed on the web page.

10. Functional Testing

Test Case	Expected Behavior
Ask a question	Verify the /qa flow returns an answer.
Get an explanation	Verify the /explain flow produces a simplified explanation.
Generate a quiz	Verify three MCQs are produced with four options each.
Summarize content	Verify long text is condensed while retaining key information.
Get a personalized learning plan	Verify a structured beginner-to-advanced recommendation is returned.

The source document records successful functional exploration of question answering, topic explanation, summarization, quiz generation, and learning recommendations.

11. User Interaction Flow

- User opens EduGenie.
- User chooses a task: Explain / QnA / Quiz / Summary / Recommend Path.
- User enters the relevant topic, question, or text.
- Frontend sends the request to the FastAPI backend.
- Backend selects the appropriate module.
- The module processes the input using the configured AI model.
- The result is returned and shown below the input area.

12. Challenges and Learning Outcomes

12.1 Development Challenges

- Maintaining accuracy and usefulness in AI-generated responses.
- Maintaining performance across devices.
- Balancing personalization with general usability.
- Handling API responses and invalid/incomplete generated output.
- Designing prompts and structured outputs that can be reliably parsed.

12.2 Learning Outcomes

- Prompt engineering and structured AI interaction.
- Cloud AI API integration.
- Modular backend architecture using FastAPI.
- User-centric interface design.
- Basic scalable application architecture and error handling.

13. Conclusion

EduGenie is a modular AI-powered educational assistant that combines cloud-based intelligence with a lightweight web interface. The completed system supports question answering, simplified explanations, quiz generation, summarization, and personalized learning paths. Its modular structure provides a clear foundation for future extensions while keeping the core application accessible and maintainable.

The project demonstrates how generative AI can be integrated into an educational workflow to make learning more interactive and easier to understand. The development process also provided practical experience in AI integration, prompt design, backend API development, frontend integration, testing, and deployment.

14. Future Enhancements

- Voice-based interaction for hands-free learning.
- Multilingual support to reduce language barriers.
- A dedicated mobile application with offline capabilities.
- Progress-tracking dashboards and learning metrics.
- Gamification through badges, learning streaks, and challenge-based assessments.
- Adaptive learning paths driven by learner progress and data.
- Group study sessions and teacher/parent dashboards.
- Integration with Learning Management Systems such as Moodle or Google Classroom.
- Image and PDF input for snapshot-based doubt solving and resource summarization.
- Real-time synchronization and smart notifications.